

# DUSY: MongoDB Integration & Storage Guide

Comprehensive Guide to Database Connectivity, Asynchronous Driver Architecture & Data Inspection

ACADEMIC VIVA  
REFERENCE

## 1. System Overview: Where & How Data Is Stored

The DUSY AI Healthcare Assistant utilizes **MongoDB Community Server (v8.0)** as its primary persistent database engine. Unlike relational databases that enforce rigid tabular schemas, MongoDB stores clinical data as binary JSON (BSON) documents, providing schema flexibility for complex healthcare telemetry.

| Component               | Technical Detail / Location                                                  |
|-------------------------|------------------------------------------------------------------------------|
| Database Engine         | MongoDB Community Server 8.0 (NoSQL Document Store)                          |
| Service Host & Port     | <code>localhost:27017</code> (Default local TCP port)                        |
| Active Database Name    | <code>health_assistant</code>                                                |
| Physical Storage Path   | <code>C:/Program Files/MongoDB/Server/8.3/data</code>                        |
| Python Async Driver     | <code>motor</code> (v3.7.0) — <code>AsyncIOMotorClient</code>                |
| Underlying BSON Library | <code>pymongo</code> (v4.12.1) — Handles <code>ObjectId</code> serialization |

## 2. Driver Selection: Why We Used Motor (AsyncIOMotorClient)

A critical technical defense point for academic evaluation is our choice of **Motor** over standard synchronous PyMongo:

- FastAPI ASGI Architecture:** FastAPI runs on an asynchronous event loop (uvicorn). Traditional PyMongo is synchronous and blocks the entire thread during disk reads/writes.
- Non-Blocking I/O:** Motor wraps MongoDB operations in non-blocking Python `asyncio` primitives ( `await db.collection.find_one(...)` ). This allows the web server to handle hundreds of concurrent requests while awaiting database responses.
- Zero Connection Overhead:** Motor manages a connection pool automatically, maintaining open socket connections across incoming HTTP requests.

## 3. Connection Lifecycle: Configuration to Shutdown

### Step 1: Configuration Loading ( `app/config.py` )

Configuration parameters are read from environment variables using Pydantic Settings:

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
```

```
MONGODB_URI: str = "mongodb://localhost:27017"
DATABASE_NAME: str = "health_assistant"
```

## Step 2: Database Singleton & Lifecycle Management ( [app/database.py](#) )

```
from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
from app.config import get_settings

class Database:
    client: AsyncIOMotorClient | None = None
    db: AsyncIOMotorDatabase | None = None

db_instance = Database()

async def connect_to_database():
    # Initializes connection pool on FastAPI application startup
    settings = get_settings()
    db_instance.client = AsyncIOMotorClient(settings.MONGODB_URI)
    db_instance.db = db_instance.client[settings.DATABASE_NAME]

async def close_database_connection():
    # Gracefully closes socket connections on shutdown
    if db_instance.client:
        db_instance.client.close()

def get_database() -> AsyncIOMotorDatabase:
    # Dependency provider for routes and services
    return db_instance.db
```

## Step 3: FastAPI Lifespan Hook ( [app/main.py](#) )

FastAPI's lifespan event guarantees database availability before any incoming request is served:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await connect_to_database()      # Connects on server boot
    yield
    await close_database_connection() # Disconnects on server shutdown
```

## 4. How MongoDB Is Utilized in Application Code

Across DUSY's services ( `auth_service.py` , `health_service.py` , `agent_service.py` ), database operations follow four primary patterns:

### A. Document Insertion ( `insert_one` )

Used during user registration and goal formulation:

```
# app/services/auth_service.py
db = get_database()
user_doc = {
    "name": user_data.name,
    "email": user_data.email.lower(),
    "password_hash": hash_password(user_data.password),
    "created_at": datetime.utcnow()
}
result = await db.users.insert_one(user_doc)
user_id = str(result.inserted_id)
```

### B. Querying & Context Assembly ( `find_one` )

Used to load the patient's verified health baseline before querying Gemini:

```
# app/services/agent_service.py
db = get_database()
profile = await db.health_profiles.find_one({"user_id": user_id})
# Yields: height_cm (185.0), weight_kg (70.0), bmi (20.5), allergies (['peanuts'])
```

### C. Upsert Operations ( `update_one` with `upsert=True` )

Ensures patient baseline vitals are updated if existing, or created on first log:

```
# app/services/health_service.py
await db.health_profiles.update_one(
    {"user_id": user_id},
    {"$set": profile_doc, "$setOnInsert": {"created_at": datetime.utcnow()}},
    upsert=True
)
```

### D. Atomic Array Appends ( `$push` )

Preserves multi-turn clinical chat transcripts without document overwrites:

```
# app/services/agent_service.py
await db.agent_consultations.update_one(
    {"_id": ObjectId(consultation_id)},
    {
        "$push": {"messages": {"$each": [user_message, agent_message]}},
        "$set": {"updated_at": datetime.utcnow()}
    }
)
```

## 5. Live Collections & Stored Data Inventory

| Collection Name     | Purpose in DUSY                      | Key Stored Attributes                                                   |
|---------------------|--------------------------------------|-------------------------------------------------------------------------|
| users               | Patient accounts & security          | _id , name , email , password_hash (Bcrypt salted)                      |
| health_profiles     | Physical baseline & vitals           | user_id , height_cm , weight_kg , bmi , blood_group , allergies         |
| agent_consultations | 5-stage clinical triage transcripts  | user_id , task_type , stage , messages array, probed_parameters         |
| agent_actions       | Confirmed lifestyle & recovery goals | user_id , action_type , title , status ('completed'), executed_at       |
| preventive_analyses | Lifestyle telemetry logs             | user_id , sleep_hours , water_intake_liters , daily_steps , risk_scores |
| conversations       | Informational Q&A sessions           | user_id , title , messages history, created_at                          |

## 6. How to View & Demonstrate Stored Data to Teachers

When your evaluators ask to see where the data is stored and how it is organized, use these two verified methods:

### Method A: Graphical Desktop Inspection (MongoDB Compass) — Recommended

- 1 Open **MongoDB Compass** from Windows Start (or search for it).
- 2 Click the green **Connect** button (connects to default `mongodb://localhost:27017` ).
- 3 From the left database panel, click on `health_assistant` .
- 4 Click on `health_profiles` or `agent_consultations` to visually show the evaluator the live JSON documents with collapsible tree structures, timestamps, and fields.

### Method B: Instant Command Line Inspection (Terminal Script)

In your project root, run the dedicated inspection script:

```
python show_database.py
```

This queries the live Motor driver and outputs:

- Physical database storage path on disk ( `C:\Program Files\MongoDB\Server\8.3\data` ).
- Document counts across all collections.
- Live patient records (User profile with **Height 185 cm, Weight 70 kg, BMI 20.5, Blood Group B+, Peanut Allergy**).
- Active clinical triage turns and committed recovery goals.

## 7. Script to Explain to Your Teacher in Viva

"Sir / Ma'am, our backend uses **MongoDB Community Server** connected via the **Motor asynchronous driver** under the database name `health_assistant` .

We specifically chose MongoDB over SQL because healthcare telemetry is polymorphic and semi-structured — different patients possess varying sets of allergies, preconditions, lifestyle metrics, and multi-turn consultation transcripts. In MongoDB, these fit naturally into native JSON documents without dozens of empty NULL columns or expensive table joins.

Motor is fully asynchronous, meaning database queries run directly on FastAPI's `asyncio` event loop without blocking server execution threads. All files are persistently stored on this machine at `C:\Program Files\MongoDB\Server\8.3\data` , and we can inspect any collection live in MongoDB Compass or through our Python inspection utility."

## 8. Common Viva Questions & Model Answers

**Q1: Why not use an ORM like SQLAlchemy with PostgreSQL?**

*Answer:* "SQLAlchemy is designed for relational tables with fixed schemas. In our AI healthcare platform, consultations have variable message depths, probe parameters, and nested action cards. MongoDB's document model represents clinical encounters and recovery goals as first-class nested structures without relational impedance mismatch."

**Q2: How do you handle password security in MongoDB?**

*Answer:* "Passwords are never stored in plaintext. In `auth_service.py`, we use **Bcrypt with Passlib** to generate salted, one-way cryptographic hashes before inserting user documents into the `users` collection."